Em Java, **wrappers** (ou *classes wrapper*) são classes especiais que **empacotam um tipo primitivo dentro de um objeto**. [1]

Como o Java não é 100% orientado a objetos (por causa dos tipos primitivos que guardam apenas o valor bruto na memória), as classes wrapper servem para transformar esses valores em objetos reais, permitindo que ganhem métodos e comportamentos extras. [1, 2, 3]

Correspondência entre primitivos e Wrappers

Para cada um dos 8 tipos primitivos do Java, existe uma classe wrapper correspondente (todas pertencem ao pacote `java.lang` e começam com letra maiúscula): [1, 2]

Tipo Primitivo	Classe Wrapper
`byte`	`Byte`
`short`	`Short`
`int`	`Integer`
`long`	`Long`
`float`	`Float`
`double`	`Double`
`char`	`Character`
`boolean`	`Boolean`

Principais diferenças e utilidades

- **Aceitam `null`:** Tipos primitivos precisam sempre ter um valor (ex: `int x = 0;`). Já as wrappers podem ser `null`, o que é muito útil para representar a ausência de um valor (como em um banco de dados ou formulário onde um campo opcional não foi preenchido). [1, 2]
- **Uso em Coleções (*Collections*):** Estruturas como `ArrayList`, `HashSet` ou `HashMap` **não** aceitam tipos primitivos. Você precisa usar wrappers (ex: `List<Integer>` em vez de `List<int>`). [1, 2, 3]

- **Métodos utilitários:** As classes wrapper trazem funções prontas para conversões e manipulação de dados, como `Integer.parseInt()` (para transformar uma `String` em número) ou constantes como `Integer.MAX_VALUE`. [1, 2]
- **Autoboxing e Unboxing:** O Java faz a conversão automática entre o tipo primitivo e o objeto wrapper sem que você precise programar isso manualmente:
  - *Autoboxing*: `Integer numero = 10;` (primitivo `int` virando objeto `Integer`).
  - *Unboxing*: `int valor = numero;` (objeto `Integer` virando primitivo `int`). [1, 2]

> **Atenção:** Como são objetos, as wrappers consomem mais memória do que os tipos primitivos e exigem cuidado ao fazer comparações diretas usando `==` (o ideal é usar `.equals()`). [1]

Mais detalhes técnicos podem ser conferidos no artigo da DevMedia ou na explicação do portal Linha de Código. [1, 2]

Aqui estão os exemplos práticos mostrando o uso de **wrappers** com `ArrayList` e o funcionamento do **Autoboxing / Unboxing**.

1. Wrappers em coleções (`ArrayList`)

Como as coleções do Java não aceitam tipos primitivos (`int`, `double`, etc.), você deve usar a classe correspondente (`Integer`, `Double`).

java

```
import java.util.ArrayList;
import java.util.List;

public class ExemploColecao {
    public static void main(String[] args) {
        // Errado: List<int> numeros = new ArrayList<>(); (Não compila)

        // Correto: Usando a classe Wrapper Integer
        List<Integer> notas = new ArrayList<>();

        // Adicionando elementos
        notas.add(85); // O Java converte o int 85 para Integer
automaticamente
        notas.add(90);
        notas.add(null); // Wrappers aceitam valor nulo!

        System.out.println("Lista de notas: " + notas);
```

Use o código com cuidado.

---

2. Autoboxing e Unboxing na prática

Esse é o mecanismo que o Java usa nos bastidores para converter o dado bruto (primitivo) em um objeto (wrapper) e vice-versa.

java

```
public class ExemploAutoboxing {
    public static void main(String[] args) {

        // --- AUTOBOXING ---
        // Converte automaticamente o primitivo 'double' em um objeto 'Double'
        double precoPrimitivo = 49.90;
        Double precoWrapper = precoPrimitivo;



        // --- UNBOXING ---
        // Converte automaticamente o objeto 'Double' de volta para o
primitivo 'double'
        Double valorObjeto = 100.0;
        double valorPrimitivo = valorObjeto;



        // --- Métodos Utilitários dos Wrappers ---
        // Exemplo: Convertendo uma String de texto em um número inteiro real
```

Use o código com cuidado.

⚠️ Um cuidado importante (NullPointerException)

Ao fazer o **Unboxing** (passar de Wrapper para Primitivo), se o seu objeto wrapper estiver guardando um valor `null`, o Java tentará extrair o valor de algo que não existe e disparará um erro no seu sistema:

java

```
Integer pontos = null;
int meusPontos = pontos; // ❌ Erro! Dispara uma NullPointerException em tempo
de execução
```

Use o código com cuidado.